BROCELIANDE TRIATHLON

# GUIDE DE GESTION DU SITE BROCÉLIANDE TRIATHLON

Ce guide s'adresse à la personne qui gère le site : comment le mettre en ligne sur GitHub, puis comment le tenir à jour au fil de la saison. Tout se fait sur le site github.com, avec un simple navigateur : il n'y a rien à installer. Le **README** (fichier `../README.md`) explique les bases (écrire dans un fichier `.json`, remplacer une photo, les outils facultatifs pour travailler sur son ordinateur). Pour modifier la mise en page ou les textes fixes des pages, voir le **guide du code** (fichier `GUIDE_CODE.md`).

Version du 30 octobre 2026 · Source : guides/GUIDE_GESTION.md, PDF refait avec npm run guide

# SOMMAIRE


**1. Comment ça marche, en une phrase**

**2. Avant la première mise en ligne**

**3. Mettre le site en ligne sur GitHub (une seule fois)**

- 3.1 Créer le compte GitHub et l'organisation du club
- 3.2 Créer le dépôt du site
- 3.3 Envoyer le site en trois paquets
- 3.4 Activer GitHub Pages, puis envoyer le dernier paquet
- 3.5 Vérifier le site en ligne
- 3.6 Plus tard : le nom de domaine broceliandetri.org
- 3.7 Les dernières publications Facebook et Instagram, automatiquement
- 3.8 Facultatif : travailler aussi sur ton ordinateur

**4. Mettre à jour le site : la routine**

- 4.1 Modifier un texte ou une liste
- 4.2 Ajouter ou remplacer une image ou un PDF
- 4.3 Modifier plusieurs fichiers en une fois : l'éditeur en ligne
- 4.4 Vérifier la publication
- 4.5 Revenir en arrière
- 4.6 Sur ton ordinateur (facultatif)

**5. Calendrier : quoi mettre à jour, quand, et dans quel fichier**

- Au fil de la saison (après une course, un article, une publication)
- À la rentrée (août – septembre)
- Après l'assemblée générale
- Triathlon de Brocéliande
- Biathlon running
- Partenaires (à chaque nouveau partenariat ou départ)
- Une fois par an (en septembre, par exemple)
- Ce qui se met à jour tout seul

**6. Où se trouve chaque texte**

**7. Photos et images**

**8. Entretien technique (une à deux fois par an)**

**9. Aide-mémoire**

- Sur github.com
- Sur un ordinateur équipé de Node.js et GitHub Desktop (facultatif)

# 1. COMMENT ÇA MARCHE, EN UNE PHRASE

Tu modifies les fichiers du dossier `src/` directement sur github.com, et GitHub fabrique puis met en ligne le site tout seul, en deux à trois minutes.




Si une erreur s'est glissée (une virgule oubliée dans un `.json`, un lien cassé), GitHub refuse de publier. **L'ancienne version reste en ligne.** Tu ne peux donc pas « casser » le site public par une faute de frappe.

Le fichier qui commande cette publication automatique est déjà dans le projet : `.github/workflows/publier.yml`. Il n'y a pas besoin d'y toucher.

# 2. AVANT LA PREMIÈRE MISE EN LIGNE

Le site est prêt : il ne contient plus aucun texte provisoire, et les pages Mentions légales et Confidentialité sont complètes. Avant de l'ouvrir au public, il reste à régler :

- **Les droits sur les photos de presse.** Les 5 photos des articles Ouest-France (accueil et Vie du club) demandent l'accord du journal. Sans accord, remplace-les par des photos du club dans `src/_data/actus.json` et `src/_data/presse.json`.
- **Ce qui sera public.** Un dépôt GitHub Pages gratuit est public : tout le monde peut lire le code, les textes et les guides du projet. Les fichiers préparés pour GitHub ne contiennent ni mot de passe, ni donnée personnelle d'adhérent, ni les éléments de travail (maquette, éléments fournis).
- **Le nom de domaine** `broceliandetri.org`, aujourd'hui chez Wix : le bureau décide quand basculer (section 3.6). En attendant, le site est en ligne à une adresse GitHub.
- **Facultatif : les publications Facebook et Instagram automatiques** (section 3.7).

# 3. METTRE LE SITE EN LIGNE SUR GITHUB (UNE SEULE FOIS)

Tout se fait sur github.com, avec un navigateur (Chrome, Edge ou Firefox) sur un ordinateur : rien à installer. Compte une heure la première fois.

## 3.1 CRÉER LE COMPTE GITHUB ET L'ORGANISATION DU CLUB

1. Crée un compte sur github.com si tu n'en as pas. Active la **double authentification** quand GitHub la propose (Settings → Password and authentication) : GitHub l'exige pour publier.
2. **Masque ton adresse e-mail** : Settings → **Emails** → coche « Keep my email addresses private ». GitHub te donne une adresse du type `12345678+pseudo@users.noreply.github.com`. C'est elle qui signera les modifications que tu feras sur github.com, à la place de ton adresse personnelle, visible de tous sur un dépôt public.
3. **Crée une organisation gratuite au nom du club**, par exemple `broceliande-triathlon` : bouton « + » en haut à droite → New organization → Free. Le site appartient ainsi au club et non à une personne. Ajoute un deuxième membre du bureau comme propriétaire (People → Invite member → Owner), pour que le site ne dépende jamais d'un seul compte.

## 3.2 CRÉER LE DÉPÔT DU SITE

1. Bouton « + » en haut à droite → **New repository**.
2. **Owner** : l'organisation du club. **Repository name** : `site-broceliande-triathlon`.
3. Choisis **Public** : GitHub Pages est gratuit pour les dépôts publics.
4. Ne demande rien en plus : « Add a README file » reste désactivé, « .gitignore » et « License » restent sur **None**. Le site contient déjà ces fichiers.
5. Clique sur **Create repository**. GitHub affiche une page « Quick setup » : le dépôt est créé, et encore vide.

## 3.3 ENVOYER LE SITE EN TROIS PAQUETS

Depuis le navigateur, GitHub accepte au plus 100 fichiers par envoi, et le site en compte 145. Ils sont donc répartis en trois paquets prêts à glisser, dans le dossier `LIVRABLES/2_DEPOT_GITHUB/envoi-en-ligne/` de ton ordinateur :

| Paquet | Contenu | Fichiers |
|---|---|---|
| `1_pages-et-donnees` | Le dossier `src` sans ses images : les pages, les textes, les listes (`.json`), l'en-tête et le pied de page | 43 |
| `2_images-et-documents` | Le dossier `src/assets` : images, PDF, vidéo, CSS et JavaScript | 75 |
| `3_reglages-et-publication` | Tout le reste : la publication automatique (dossier `.github`), les guides, les outils, `README.md`, `package.json` | 27 |

Envoie-les **dans l'ordre**. Pour chacun, la manœuvre est la même :

1. **Ouvre la page d'envoi** :
   - pour le paquet 1, sur la page « Quick setup » du dépôt vide, clique sur le lien **uploading an existing file** ;
   - pour les paquets suivants, sur la page du dépôt, bouton **Add file** → **Upload files**.
2. Dans l'Explorateur de fichiers, **ouvre le dossier du paquet**, sélectionne tout son contenu (**Ctrl + A**) et **fais-le glisser** sur la page d'envoi. Le bouton « choose your files » ne sait pas choisir des dossiers : il faut glisser.
3. Attends que la liste des fichiers ait fini de s'afficher sous la zone d'envoi.
4. En bas de la page, dans « Commit changes », écris un résumé (par exemple « Envoi 1 : pages et données »), laisse « Commit directly to the main branch » coché, puis clique sur **Commit changes**.
5. Vérifie sur la page du dépôt que le dossier `src` apparaît (paquet 1), puis qu'il contient `assets` (paquet 2).

**Arrête-toi après le paquet 2** : avant d'envoyer le paquet 3, qui lance la publication, il faut activer GitHub Pages.

## 3.4 ACTIVER GITHUB PAGES, PUIS ENVOYER LE DERNIER PAQUET

1. Sur la page du dépôt : **Settings** → **Pages**.
2. Dans « Build and deployment » → **Source**, choisis **GitHub Actions**. GitHub propose alors des modèles (« Jekyll », « Static HTML »…) : n'en choisis aucun, le site a déjà le sien.
3. Envoie le **paquet 3**, comme les deux premiers (section 3.3).
4. Vérifie que la liste des fichiers du dépôt montre bien les quatre noms qui commencent par un point : `.github`, `.gitattributes`, `.gitignore` et `.htmlvalidate.json`. S'il en manque un, crée-le à la main : **Add file** → **Create new file**, tape son nom complet (par exemple `.github/workflows/publier.yml` : chaque barre oblique crée un dossier), colle son contenu (ouvre le fichier du paquet avec le Bloc-notes, puis Ctrl + A et Ctrl + C), puis **Commit changes**.
5. Onglet **Actions** : la publication « Publier le site » démarre toute seule. Après deux à trois minutes, une coche verte apparaît. Le site est en ligne à l'adresse affichée dans Settings → Pages, du type : `https://broceliande-triathlon.github.io/site-broceliande-triathlon/`
6. Si rien ne démarre, ou en cas de croix rouge : onglet **Actions** → « Publier le site » → **Run workflow** → **Run workflow**.

Le site sait fonctionner dans ce sous-dossier : les liens, images, polices et la vidéo s'y adaptent automatiquement.

**Important :** à partir de maintenant, **le site, c'est le dépôt sur github.com**. Toutes les modifications s'y font (section 4). Les dossiers de ton ordinateur (`PROJET SITE BT`, `LIVRABLES`) ne sont plus que des archives : ne renvoie plus les paquets.

## 3.5 VÉRIFIER LE SITE EN LIGNE

Ouvre l'adresse du site sur un ordinateur **et** sur un téléphone, et vérifie :

- le menu, chaque page, et les liens du pied de page ;
- les documents PDF de la page Nous rejoindre et de Nos évènements ;
- le bandeau Facebook/Instagram : « Accepter » affiche les publications, « Refuser » n'affiche rien ;
- la vidéo du triathlon, qui se lance au clic ;

- le bouton « Envoyer » de la page Contact, qui ouvre ta messagerie avec le message prérempli.

Si quelque chose ne va pas, corrige-le directement sur github.com (section 4).

## 3.6 PLUS TARD : LE NOM DE DOMAINE BROCELIANDETRI.ORG

Quand le bureau aura décidé de quitter Wix :

1. Dans le dépôt : **Settings** → **Pages** → **Custom domain** → saisis `www.broceliandetri.org` → Save.
2. Chez le gestionnaire du nom de domaine (Wix si le domaine y a été acheté, ou un autre registraire après transfert), règle les DNS :
   - `www` : enregistrement **CNAME** vers `broceliande-triathlon.github.io` (le nom de l'organisation ou du compte).
   - le domaine nu `broceliandetri.org` : 4 enregistrements **A** vers `185.199.108.153`, `185.199.109.153`, `185.199.110.153` et `185.199.111.153`.
3. Quand GitHub l'indique (quelques minutes à quelques heures), coche **Enforce HTTPS**.
4. Vérifie que `src/_data/site.json` indique bien `"url": "https://www.broceliandetri.org"`.

À ce moment-là, le site Wix cesse d'être affiché. Prévois la bascule à un moment calme, hors période d'inscriptions.

## 3.7 LES DERNIÈRES PUBLICATIONS FACEBOOK ET INSTAGRAM, AUTOMATIQUEMENT

Par défaut, le site affiche les publications dont tu as copié les liens dans `src/_data/publications.json`. Il peut aussi aller chercher tout seul les 3 dernières publications de la page Facebook et du compte Instagram : le site est alors régénéré **chaque nuit** par GitHub. Il faut pour cela trois clés d'accès fournies par Meta, à préparer une seule fois avec le compte Facebook d'un administrateur de la page du club.

1. **Instagram** : passe le compte `@broceliandetriathlon` en **compte professionnel** (Paramètres → Type de compte et outils), puis relie-le à la page Facebook du club (Centre de comptes, ou Paramètres de la page → Comptes liés).
2. Sur developers.facebook.com, connecte-toi avec ton compte Facebook, puis **Mes apps** → **Créer une app**. Choisis un usage de type « Autre », puis « Entreprise ». Donne-lui un nom, par exemple « Site Brocéliande Triathlon ».
3. Ouvre l'**Explorateur de l'API Graph** (menu Outils). Choisis ton app, puis **Générer un jeton d'accès** en cochant les autorisations `pages_show_list`, `pages_read_engagement` et `instagram_basic`. Accepte les fenêtres de Facebook en choisissant la page du club.
4. Ouvre le **Débogueur de jeton d'accès** (menu Outils), colle ce jeton, puis clique sur **Prolonger le jeton d'accès** : tu obtiens un jeton longue durée.
5. Retourne dans l'Explorateur avec ce jeton longue durée et lance la requête `me/accounts`. Dans la réponse, repère la page du club :
   - `id` : c'est l'**identifiant de la page** ;
   - `access_token` : c'est le **jeton de la page**. Celui-ci n'expire pas, sauf si l'administrateur change de mot de passe ou perd son rôle sur la page.
6. Lance la requête `IDENTIFIANT_DE_LA_PAGE?fields=instagram_business_account` : le nombre indiqué est l'**identifiant Instagram**.

7. Sur GitHub, dans le dépôt : **Settings** → **Secrets and variables** → **Actions** → **New repository secret**, crée trois secrets :
   - `META_JETON_PAGE` : le jeton de la page ;
   - `META_ID_PAGE` : l'identifiant de la page ;
   - `META_ID_INSTAGRAM` : l'identifiant Instagram.
8. Onglet **Actions** → **Publier le site** → **Run workflow**. Dans le journal de l'étape « Générer le site pour son adresse de publication », tu dois lire « Facebook : 3 publication(s) récupérée(s) » et « Instagram : 3 publication(s) récupérée(s) ».

Bon à savoir :

- **Ces clés sont secrètes.** Ne les écris jamais dans un fichier du projet : le dépôt est public. Les secrets GitHub ne sont visibles par personne, même dans les journaux.
- **En cas de problème** (jeton expiré, Meta indisponible), le site ne tombe pas en panne : il reprend les listes de `publications.json`, et le journal l'indique. Garde donc ces listes à peu près à jour.
- **GitHub met en pause** les publications nocturnes d'un dépôt sans aucune modification pendant 60 jours. Une simple modification enregistrée les relance.

## 3.8 FACULTATIF : TRAVAILLER AUSSI SUR TON ORDINATEUR

Rien n'oblige à installer quoi que ce soit : tout le site se gère sur github.com. Deux outils gratuits apportent du confort pour les gros changements (nouvelle section, mise en page) :

- **Node.js** fabrique le site sur ton ordinateur. Tu vois le résultat avant de publier (`npm run dev`), tu lances la vérification complète, accessibilité comprise (`npm run check`), et tu as accès aux outils du projet : PDF de la check-list et des guides, fichier de relecture.
- **GitHub Desktop** fait le lien entre ton ordinateur et le dépôt du site.

Pour les mettre en place, une fois le site en ligne :

1. Installe Node.js (version « LTS ») et GitHub Desktop. Dans GitHub Desktop, connecte-toi avec ton compte, puis vérifie que l'adresse e-mail utilisée est bien l'adresse `noreply` de l'étape 3.1 (File → Options → Git).
2. **File** → **Clone repository** → choisis `site-broceliande-triathlon` dans la liste de l'organisation du club → choisis l'emplacement (par exemple `Documents`) → **Clone**.
3. Ouvre un terminal dans ce dossier (voir le README, section 1), puis installe les outils du site avec `npm install`, et vérifie avec `npm run check`.

Ensuite, suis la méthode « Sur ton ordinateur » de la section 4.6.

Bon à savoir : le premier envoi peut aussi se faire avec GitHub Desktop, à la place des sections 3.2 et 3.3. Dans ce cas, ne crée pas le dépôt sur github.com : **File** → **Add local repository** → dossier `LIVRABLES/2_DEPOT_GITHUB/site-broceliande-triathlon` → **create a repository** (Git ignore et License : None) → **Publish repository** (organisation du club, « Keep this code private » décoché). Active ensuite GitHub Pages (section 3.4, étapes 1, 2, 5 et 6).

# 4. METTRE À JOUR LE SITE : LA ROUTINE

Toutes les modifications se font sur github.com, dans le dépôt du site. Chaque modification enregistrée (« Commit changes ») est vérifiée, puis publiée toute seule en deux à trois minutes. Deux raccourcis utiles sur la page du dépôt : la touche **t** cherche un fichier par son nom, la touche **.** (point) ouvre l'éditeur en ligne (section 4.3).

## 4.1 MODIFIER UN TEXTE OU UNE LISTE

1. Dans le dépôt, ouvre le fichier, par exemple `src/_data/agenda.json` : clique sur `src`, puis sur `_data`, puis sur le fichier.
2. Clique sur l'icône **crayon** (Edit this file).
3. Fais la modification. Les règles d'écriture des fichiers `.json` sont dans le README, section 3.
4. Clique sur **Commit changes...**, écris un résumé (par exemple « Agenda : date du biathlon 2027 »), laisse « Commit directly to the main branch » coché, puis clique sur **Commit changes**.

## 4.2 AJOUTER OU REMPLACER UNE IMAGE OU UN PDF

1. Ouvre le dossier : `src/assets/img/` pour les images, `src/assets/docs/` pour les PDF.
2. Bouton **Add file** → **Upload files**, glisse le fichier sur la page, puis **Commit changes**.
3. Un fichier envoyé avec **exactement le même nom** qu'un fichier existant le remplace : le site l'utilise aussitôt. Avec un nouveau nom, indique ensuite ce nom dans le fichier `.json` concerné (section 4.1). Envoie toujours le fichier **avant** de modifier le `.json` : un `.json` qui cite une image absente bloque la publication.

Pour supprimer un fichier : ouvre-le, menu **...** en haut à droite → **Delete file** → **Commit changes**.

## 4.3 MODIFIER PLUSIEURS FICHIERS EN UNE FOIS : L'ÉDITEUR EN LIGNE

Sur la page du dépôt, appuie sur la touche **.** (point) : un éditeur complet s'ouvre dans le navigateur (son adresse commence par `github.dev`). Tu y ouvres et modifies plusieurs fichiers à la suite, et une erreur de virgule ou de guillemet dans un `.json` y est soulignée en rouge. C'est aussi l'outil le plus sûr pour une grosse modification d'un `.json`.

Pour enregistrer : icône **Source Control** (dans la barre de gauche, celle qui porte le nombre de fichiers modifiés) → écris un résumé → **Commit & Push**. Pour revenir au dépôt : menu ☰ en haut à gauche → **Go To Repository**.

## 4.4 VÉRIFIER LA PUBLICATION

- Onglet **Actions** du dépôt : une ligne par modification. Rond orange = en cours ; coche verte = en ligne ; croix rouge = publication refusée.

- **En cas de croix rouge**, rien n'est cassé : l'ancienne version reste en ligne. Ouvre la ligne, puis l'étape marquée en rouge : le message indique le fichier, et souvent la ligne, en faute. Il s'agit presque toujours d'une virgule ou d'un guillemet dans un `.json`. Corrige le fichier (section 4.1) : la publication repart toute seule.
- Après la coche verte, ouvre la page modifiée sur le site en ligne, sur ordinateur et sur téléphone. Si l'ancienne version s'affiche encore, recharge la page (Ctrl + F5 sur ordinateur).

## 4.5 REVENIR EN ARRIÈRE

GitHub garde toutes les versions de chaque fichier. Pour remettre un fichier comme avant :

1. Ouvre le fichier sur github.com, puis clique sur **History** (en haut à droite) : ses versions s'affichent, la plus récente en haut.
2. Sur la ligne de la version à retrouver, clique sur l'icône **<>** (Browse repository at this point) : le dépôt s'affiche tel qu'il était ce jour-là. Rouvre le fichier si besoin.
3. Clique sur l'icône **Copy raw file** (deux carrés superposés, au-dessus du contenu) : tout l'ancien contenu est copié.
4. Reviens au fichier actuel (lien du dépôt en haut de la page), crayon, **Ctrl + A** puis **Ctrl + V**, et **Commit changes**.

Pour une image ou un PDF : à l'étape 3, clique sur **Download raw file**, puis renvoie le fichier téléchargé sous le même nom (section 4.2).

## 4.6 SUR TON ORDINATEUR (FACULTATIF)

Si tu as installé Node.js et GitHub Desktop (section 3.8) :

1. **Récupère la dernière version** avant toute chose : GitHub Desktop → **Fetch origin**, puis **Pull**. Les modifications faites sur github.com n'arrivent pas toutes seules sur ton ordinateur.
2. **Modifie** les fichiers (voir le calendrier, section 5).
3. **Regarde le résultat** : `npm run dev` → http://localhost:8080.
4. **Vérifie** : `npm run check`. Tout doit finir par des coches ✓.
5. **Envoie** : dans GitHub Desktop, écris un résumé, puis **Commit to main** et **Push origin**.
6. **Contrôle** l'onglet Actions sur GitHub (section 4.4).

Pour annuler une modification depuis GitHub Desktop : onglet **History** → clic droit sur l'enregistrement fautif → **Revert changes in commit** → **Push origin**.

# 5. CALENDRIER : QUOI METTRE À JOUR, QUAND, ET DANS QUEL FICHIER

Tous les fichiers `.json` sont dans `src/_data/`. Les images vont dans `src/assets/img/` et les PDF dans `src/assets/docs/`.

## AU FIL DE LA SAISON (APRÈS UNE COURSE, UN ARTICLE, UNE PUBLICATION)

| Quoi | Fichier | Remarques |
|---|---|---|
| Les cartes « L'actu fraîche » de l'accueil | `actus.json` | 2 ou 3 cartes (une ligne sur ordinateur). Mettre la plus récente en premier. Pour une photo de presse : `credit` (« Photo : Nom / Ouest-France ») et accord du journal. |
| Revue de presse (Vie du club) | `presse.json` | Ajouter le nouvel article en haut. `couleur` : `rose`, `orange` ou `noir`. |
| Publications Facebook intégrées (accueil et Vie du club : les 3 premières, qui défilent) | `publications.json` → `facebookIntegrations` | Liste des liens des publications à afficher avec le code d'intégration officiel de Facebook, après accord du visiteur (cookies Meta). Sur Facebook, clique sur la date de la publication pour l'ouvrir seule, puis copie l'adresse de la barre du navigateur (`https://www.facebook.com/broceliandetri/posts/…`). Mets la plus récente en premier. Le fil complet de la page n'est plus utilisé : Facebook ne l'affiche plus aux visiteurs non connectés. Si les clés de Meta sont en place (section 3.7), les 3 dernières publications sont récupérées automatiquement chaque nuit et cette liste ne sert plus que de secours. |
| Vignettes Facebook (accueil : 6, Vie du club : les 4 premières) | `publications.json` → `facebook` | Ce sont elles que voient les visiteurs qui n'ont pas accepté les contenus Facebook et Instagram : mets-les à jour de temps en temps. Enregistre l'image de la publication dans `src/assets/img/` (par exemple `fb_2027-03-vannes.jpg`), puis renseigne `image`, `alt`, `date`, `legende` (un extrait court) et `url` (le lien de la publication). Une vignette dont la date ou la légende est encore entre crochets n'est pas affichée. |
| Vignettes Instagram (Vie du club : 6) | `publications.json` → `instagram` | Même principe : image, `alt`, lien de la publication. Ce sont elles que voient les visiteurs qui n'ont pas accepté les contenus Facebook et Instagram. |

| Quoi | Fichier | Remarques |
|---|---|---|
| Publications Instagram intégrées (Vie du club : les 3 premières, qui défilent) | `publications.json` → `instagramIntegrations` | Liste des liens des publications à afficher avec le code d'intégration officiel d'Instagram, après accord du visiteur (cookies Meta). Copie le lien d'une publication (menu « … » → Copier le lien), sous la forme `https://www.instagram.com/p/XXXXXXXXXXX/`, et mets la plus récente en premier. Les 3 premières de la liste s'affichent et défilent avec des flèches. Instagram ne propose pas de fil automatique : c'est toi qui choisis les publications. Même automatisation possible que pour Facebook (section 3.7). |
| Agenda (frise horizontale de Vie du club, verticale sur l'accueil) | `agenda.json` | Ranger les rendez-vous dans l'ordre chronologique et retirer les dates passées de temps en temps. `jour` et `mois` sont le texte affiché (par exemple `FIN` et `JANV.`). `date` sert à la frise : `AAAA-MM-JJ` (`2027-05-23`), ou `AAAA-MM` si seul le mois est connu, vide si la date est inconnue. Les rendez-vous passés sont grisés et la barre avance toute seule. `couleur` : `noir`, `orange` ou `rose`. Ajouter `"accueil": false` à un rendez-vous pour le garder sur Vie du club seulement. |
| Vidéo du triathlon (accueil, sous l'ouverture, et page Nos évènements) | `evenements.json` → `triathlon.video` | `url` : lien de la vidéo Facebook ; `titre`, `duree`, `auteur` (crédit affiché sous la vidéo) ; `affiche` : image d'accroche dans `src/assets/img/` (une image de la vidéo, au format 16:9). Le lecteur de Facebook ne se charge qu'au clic du visiteur. Pour retirer la vidéo, supprimer le bloc `video`. |

## À LA RENTRÉE (AOÛT - SEPTEMBRE)

| Quoi | Fichier |
|---|---|
| Tarifs des licences | `tarifs.json` |
| Documents à télécharger (bulletin, charte…) | PDF dans `src/assets/docs/`, puis `documents.json` (champ `fichier` = nom du PDF ; changer aussi l'année dans le titre) |
| Saison affichée en haut de la page Nous rejoindre (« Adhésions 2026 – 2027 ») et texte sur les permanences d'inscription | `src/nous-rejoindre.njk` |
| Horaires des entraînements | `disciplines.json` (les 4 cartes) et `semaine.json` (la semaine type, pour chaque saison) |
| Lieux d'entraînement | `lieux.json` |

| Quoi | Fichier |
|---|---|
| Équipement : saison et tenues | `tenues.json` : `saison`, puis les `disciplines` (titre, couleur, pictos), chacune avec ses `groupes` (par exemple « Noir » et « Blanc ») et leurs `articles` ( `nom`, `image` ). Photos sur fond blanc dans `src/assets/img/` (section 4.2), envoyées avant de modifier `tenues.json`. À la publication, chaque nouveau visuel est recadré tout seul pour que l'article tienne entier dans le losange (sur ton ordinateur, `npm run tenues` fait la même chose). |
| Horaires de baignade de la piscine Océlia (fenêtre de la page Entraînements) | Remplacer l'image `src/assets/img/horaires_piscine_ocelia.png` en gardant ce nom |

## APRÈS L'ASSEMBLÉE GÉNÉRALE

| Quoi | Fichier |
|---|---|
| Bureau, rôles et membres | `bureau.json` |
| Nombre de licenciés, chiffres clés | `club.json` (accueil et page Le club) |
| Date et lieu de la prochaine AG | `agenda.json` |
| Président, directeur de la publication, siège | `legal.json` (mentions légales et confidentialité) |

## TRIATHLON DE BROCÉLIANDE

| Quand | Quoi | Fichier |
|---|---|---|
| Dès que la date de l'édition suivante est connue | Date et heure de départ (le compte à rebours J-xxx se calcule tout seul), édition, textes | `evenements.json` → `triathlon` ( `depart`, `surtitreAccueil`, `surtitre`, `resumeAccueil`, `texte` ) |
| À l'ouverture des inscriptions | Liens des boutons Inscriptions, Règlement, Check-list (un bouton sans lien reste visible mais inactif) | `evenements.json` → `triathlon.boutons` (champ `url` ) |
| Nouveaux parcours | Plans des parcours (PDF dans `src/assets/docs/` ) et tracés Garmin Connect. Tant que `afficher` vaut `false`, la page affiche seulement `messageAVenir` (« Le parcours 2027 sera communiqué… ») ; passer `afficher` à `true` pour montrer les liens | `evenements.json` → `triathlon.parcours` |
| Si la check-list du triathlète change | Le contenu de la check-list (étapes et éléments). Le pense-bête de la page Nous rejoindre suit tout seul. Le PDF à imprimer, lui, se refait sur un ordinateur équipé de Node.js ( `npm run checklist` , section 3.8) ; en attendant, l'ancien PDF reste en ligne | `checklist.json` |

| Quand | Quoi | Fichier |
|---|---|---|
| Nouvelle affiche | Image dans `src/assets/img/`, puis `affiche` | `evenements.json` |
| Après la course | Lien des résultats Klikego (ajouter en haut), et « Rappel de l'édition » avec les horaires de l'édition passée | `resultats.json`, `evenements.json` → `rappel` |

## BIATHLON RUNNING

| Quoi | Fichier |
|---|---|
| Date, textes, boutons Klikego et règlement, affiche, rappel de l'édition précédente | `evenements.json` → `biathlon` |
| Date dans l'agenda | `agenda.json` |

## PARTENAIRES (À CHAQUE NOUVEAU PARTENARIAT OU DÉPART)

| Quoi | Fichier |
|---|---|
| Partenaires principaux (logo, activité, description, lien) | `partenaires.json` → `principaux`, logo dans `src/assets/img/` |
| Liste « Merci à tous nos autres partenaires » | `partenaires.json` → `autres` |
| Collectivités | `collectivites.json` |

## UNE FOIS PAR AN (EN SEPTEMBRE, PAR EXEMPLE)

- Vérifier qu'aucun texte provisoire entre crochets n'est resté : onglet **Actions** → dernière publication → « Générer et vérifier le site » → étape « Repérer les textes provisoires entre crochets ». Elle doit finir par « 0 texte(s) entre crochets. » (sur ton ordinateur : `npm run build` puis `npm run crochets`).
- Relire les pages légales (`mentions-legales.njk`, `confidentialite.njk`), surtout si l'hébergeur ou le formulaire de contact change.
- Mettre à jour les outils (voir section 8).

## CE QUI SE MET À JOUR TOUT SEUL

- L'année du « © » en pied de page.
- Le compte à rebours « J-xxx » (calculé à partir de `triathlon.depart`).
- La frise de l'agenda : rendez-vous passés grisés et repère « Aujourd'hui » (calculés à partir du champ `date` de `agenda.json`).
- Le plan du site (`sitemap.xml`) et les versions allégées des images (AVIF, WebP).
- Le recadrage des nouveaux visuels de tenues, dans leur losange.
- Les informations pour les moteurs de recherche (le club, la date et le lieu du triathlon), tirées de `site.json`, `reseaux.json` et `evenements.json`.

# 6. OÙ SE TROUVE CHAQUE TEXTE

Les listes sont dans les fichiers `.json`. Les textes fixes de chaque page sont dans les fichiers `.njk` du dossier `src/`. Dans ces fichiers, ne modifie que le texte entre les balises, par exemple ce qui se trouve entre `<p>` et `</p>`.

| Page | Fichier des textes | Données utilisées |
|---|---|---|
| Accueil | `src/index.njk` | `actus`, `publications`, `club`, `evenements`, `agenda` |
| Le club | `src/le-club.njk` | `club`, `bureau`, `tenues` |
| Nous rejoindre | `src/nous-rejoindre.njk` | `tarifs`, `documents`, `checklist` (pense-bête) |
| Entraînements | `src/entrainements.njk` | `disciplines`, `semaine`, `lieux` |
| Vie du club | `src/vie-du-club.njk` | `presse`, `publications`, `agenda`, `reseaux` |
| Nos évènements | `src/evenements.njk` | `evenements`, `resultats` |
| Partenaires | `src/partenaires/index.njk` | `partenaires`, `collectivites` |
| Devenir partenaire | `src/partenaires/devenir-partenaire.njk` | |
| Contact | `src/contact.njk` | `lieux`, `reseaux`, `site` |
| En-tête et pied de page | `src/_includes/composants/` | `navigation`, `reseaux`, `site` |

# 7. PHOTOS ET IMAGES

- **Où :** `src/assets/img/`. Le site crée tout seul les versions légères.
- **Taille :** 2 000 pixels de large au maximum. JPG pour les photos, PNG pour les logos.
- **Noms de fichiers :** sans espace ni accent, par exemple `fb_2027-03-vannes.jpg`.
- **Texte alternatif (`alt`) :** une phrase qui décrit la photo, pour les personnes aveugles et pour le référencement.
- **Droits :** une photo trouvée sur internet n'est pas libre de droits. Pour les photos de presse ou de photographes, demande l'accord et indique le crédit.
- **Ménage :** une image qui n'est plus citée dans aucun fichier n'est pas publiée. Tu peux la supprimer du dossier pour l'alléger.

# 8. ENTRETIEN TECHNIQUE (UNE À DEUX FOIS PAR AN)

Le site est fait de simples fichiers : les outils du projet ne servent qu'à le fabriquer, sur les serveurs de GitHub. Rien ne presse donc. Seule la mise à jour des outils demande un ordinateur équipé de Node.js ; le reste se fait sur github.com, ou tout seul.

- **Node.js :** sur GitHub, la publication utilise toute seule la dernière version « LTS » de Node.js. Rien à faire.
- **Les outils du projet** (Eleventy et ses compagnons, listés dans `package.json`) : une fois par an, sur un ordinateur équipé de Node.js (section 3.8), ou avec l'aide d'un bénévole qui l'est.

  1. Pour voir ce qui a évolué :

     ```
     npm outdated
     ```

  2. Pour installer les mises à jour mineures :

     ```
     npm update
     ```

  3. Puis vérifie que tout fonctionne encore :

     ```
     npm run check
     ```

  Si tout passe, envoie le changement sur GitHub (section 4.6). Pour une version majeure (premier chiffre qui change), mieux vaut se faire aider. Sans mise à jour, le site continue simplement d'être fabriqué avec les versions actuelles.

- **GitHub Actions :** si un avertissement « deprecated » apparaît dans l'onglet Actions, il faut mettre à jour les numéros de version (`@v4`, `@v5`…) dans `.github/workflows/publier.yml`. Cela se fait directement sur github.com, avec le crayon (section 4.1).

# 9. AIDE-MÉMOIRE

## SUR GITHUB.COM

| Je veux… | Je fais… |
|---|---|
| Modifier un texte ou une liste | Fichier → crayon → **Commit changes** (section 4.1) |
| Ajouter ou remplacer une image, un PDF | Dossier → **Add file** → **Upload files** → glisser → **Commit changes** (section 4.2) |
| Modifier plusieurs fichiers d'un coup | Touche **.** sur la page du dépôt → **Source Control** → **Commit & Push** (section 4.3) |
| Retrouver un fichier | Touche **t** sur la page du dépôt, puis son nom |
| Savoir si la publication a réussi | Onglet **Actions** : coche verte = en ligne, croix rouge = à corriger (l'ancienne version reste en ligne) |
| Annuler une modification | Fichier → **History** → ancienne version → **Copy raw file** → crayon → coller → **Commit changes** (section 4.5) |
| Lister ce qui reste entre crochets | **Actions** → dernière publication → étape « Repérer les textes provisoires entre crochets » |
| Ajouter ou changer une tenue | Visuel sur fond blanc dans `src/assets/img/`, puis article dans `src/_data/tenues.json` : le recadrage se fait à la publication |

## SUR UN ORDINATEUR ÉQUIPÉ DE NODE.JS ET GITHUB DESKTOP (FACULTATIF)

| Je veux… | Je fais… |
|---|---|
| Récupérer la dernière version | GitHub Desktop : **Fetch origin** → **Pull** (toujours en premier) |
| Voir le site avant de publier | `npm run dev` puis http://localhost:8080 |
| Vérification complète, accessibilité comprise | `npm run check` |
| Publier mes changements | GitHub Desktop : résumé → **Commit to main** → **Push origin** |
| Annuler une modification publiée | GitHub Desktop → History → **Revert changes in commit** → **Push origin** |
| Refaire le PDF de la check-list | Modifier `src/_data/checklist.json`, puis `npm run checklist` |
| Refaire les PDF des guides | `npm run guide` (les guides se lisent aussi directement sur github.com, dossier `guides/`) |

| Je veux… | Je fais… |
| --- | --- |
| Envoyer une version de relecture par e-mail | `npm run html` (fichier dans `LIVRABLES/3_VERSIONS_RELECTURE/` ) |